Job Watcher Project

Financial Data Analytics

Overview

You are building a tool that watches the job market for you. Every day, on its own, the tool pulls the open positions at the employers you are interested in, compares them with what it saw the day before, records what is new and what is gone, and publishes the result to a website.

Projects are 40% of your course grade, and the project grades are averaged.

You will build the tool in a public GitHub repository, run it with GitHub Actions, and publish the site with GitHub Pages. You will turn in one text file (`.txt` or `.md`) with four lines: your name, your student ID, the repository URL, and the live site URL.

The first-run date and the due date are posted on Blackboard. By the first-run date, the pipeline should have completed its first successful run. By the due date, the pipeline should have completed a successful run on at least seven different days. No late submissions are allowed.

Watch List

Choose a role in any field, for example financial analyst, data analyst, FP&A, business intelligence analyst, credit analyst, supply chain analyst, or investment banking analyst. Then choose the employers the tool watches. The sources should meet the following requirements:

1. At least 15 employers. The tool should read each employer's job board through the applicant tracking system that hosts it (Workday, Oracle, Greenhouse, Lever, Ashby, SmartRecruiters, and others).
2. Every employer's job board should return JSON to a request made with no login. Do not read the HTML of a web page.

If an employer blocks the tool or has no job board the tool can read, replace that employer with another. If a board returns more than 5,000 postings, the tool should request it with a search term, and the README should say which term.

Pipeline

The pipeline is a Python program that GitHub Actions runs once a day on a schedule. Each run should do the following:

1. Call every source and save what it returns, compressed, in `data/raw/<date>/<source>.json.gz`. One source failing should not stop the run. A source that fails writes no raw file that day, and the result of every source, success or failure, should be recorded.
2. Build the history file from the raw files, using the steps below. The history file should depend on the raw files alone, so that running the build again produces the same file and a change to a rule applies to every day of the history.
3. Write the status file.
4. Open a GitHub Issue titled with the date that lists the new postings and the removed postings, each with a link, and close the previous day's issue. The issue should be opened every day, even on a day with no changes.
5. Commit the raw files, the history file, and the status file to the repository.

A successful run is a run that finishes and commits, even when some sources failed. A second run on the same day should replace that day's raw files, build the history again, and update that day's issue, so that each day counts once.

The build should do the following with the raw files, day by day:

1. Convert every posting from every source into one common set of columns. Keep every posting the employer's board returns, not only the ones in your role.
2. Mark each posting as in your role or not, using a title rule you write.
3. Update the history. A posting seen for the first time gets that day as its first-seen date. Every posting its source lists that day gets that day as its last-seen date. An open posting is marked removed, with that day's date, when its source has a raw file for that day and no longer lists it. A removed posting that its source lists again becomes open again, with its removed date cleared and its first-seen date unchanged. The rows are sorted by `posting_id`.

Data

The pipeline should keep two files at fixed paths.

`data/history.csv` has one row per posting and the following columns: `posting_id` (the source and the source's own ID for the posting), `source`, `source_id`, `employer`, `title`, `location`, `department`, `posted_date`, `url`, `in_role` (`true` or `false`), `first_run` (`true` for a posting listed in the first raw file of its source), `first_seen`, `last_seen`, `status` (`open` or `removed`), and `removed_date`. Dates are UTC dates written as `YYYY-MM-DD`. Every run takes the UTC date of the moment it runs. A column that a source does not provide, such as `posted_date` or `department`, is left empty for that source's rows.

`data/status.json` describes the latest run and has the following keys: `run_time` (UTC), `first_run_date` (the date of the earliest raw file), `days_with_success` (the number of different dates with a raw file), `open` (the number of open postings), `new_today` (postings first seen in the latest run, leaving out first-run postings), `removed_today` (postings removed in the latest run), and `sources`, a list with one entry per source holding `name`, `ok` (`true` or `false`), and `count`.

Site

The site should have three pages that load the two data files in the browser and do the calculations there.

`index.html` is the Today page, the page that opens at the site URL. It should show the time of the last run, the status of every source, the four counts from the status file (days with a successful run, open, new today, removed today), and the lists of new and removed postings in the latest run.

`browse.html` is the Browse page. It should show the open postings with the following:

1. A switch between postings in your role, which is the default, and all postings.
2. Filters for employer, location, and days open.
3. A search box that matches titles.
4. A sort by first-seen date.
5. A link to apply for every posting.
6. A star on every posting that saves it to a shortlist. The shortlist should be stored in the browser and still be there after the page is reloaded.

`market.html` is the Market page, a report on the history the pipeline has built. It should compute every number in the browser from the history file, so that the page changes as the history grows. It should hold the following six sections, each with a heading that states the finding, a paragraph that explains it, and a chart. Postings with `first_run` set to `true` were already open when the tool started watching their source, so their start date is unknown; the definitions below leave them out where noted.

1. New postings per day: postings first seen on each day, leaving out first-run postings, shown for all postings and for your role.
2. Removed postings per day: postings with a removed date on each day.
3. Open postings per employer as of the last run.
4. Days open: for removed postings that are not first-run postings, the removed date minus the first-seen date. The section reports the median.
5. Share removed within seven days: among postings that are not first-run postings and were first seen at least seven days before the last run, the share whose removed date minus first-seen date is seven days or less.
6. Top locations among open postings.

The site should be installable. It should have a web app manifest and a service worker, and it should install from the browser as an app. When the computer is online, the pages load the data files from the network. When it is offline, the site still opens and shows the last data it loaded.

All three pages should share one navigation bar and one set of fonts and colors. The look is up to you.

Tests

The repository should have a test suite written with pytest. The tests should cover the conversion of each source to the common columns, the title rule, and the first-seen, last-seen, and removed logic. GitHub Actions should run the tests on every push, and the README should show the status badge.

Repository

Build the project in its own folder under your home folder, for example `~/job-watcher`. That folder is the repository. Do not build it inside the `~/fda-python` course folder.

The repository should be public, with GitHub Pages enabled, and should hold the following:

- A `README.md` that lists every file and what it does, lists every source with its applicant tracking system, states the title rule, and gives the command that builds the history file from the raw files.
- The pipeline program, the tests, the workflow files, the data files, and the site files.
- A commit history that shows the work as it was done, not one commit holding the finished project. The daily commits made by the pipeline are part of that history.

Using AI Tools

You are expected to build the tool with Claude Code or Codex.

Grading

Every requirement is a minimum. More employers, another applicant tracking system, an aggregator with an API key, a different way to be notified, more pages or more charts are all welcome.

The project is graded out of 100 points.

Component	Points
Pipeline: daily runs, source status, survives a failed source	25
Data: history correct and built from the raw files	20
Site: three pages that work, filters, charts, installable	25
Market: numbers follow the definitions and agree with the history file	10
Tests: run and pass on every push	10
Repository: README, data files at the fixed paths, Issues, commit history	10

A pipeline with successful runs on fewer than seven different days, or whose last successful run is more than 48 hours before the due date, earns zero credit for the pipeline component. A history file that the build does not reproduce from the raw files earns zero credit for the data component. A Market page whose numbers do not match the history file earns zero credit for the Market component. A site that does not work at the site URL earns zero credit for the site component.

Academic Integrity

The tool is your own work. You may discuss employers, sources, and publishing problems with classmates, but do not share code or files.